

Independent Spanning Trees on Cartesian Product of Hybrid Graphs*

Jinn-Shyong Yang[†] Jou-Ming Chang

Institute of Information and Decision Sciences,
National Taipei College of Business, Taipei, Taiwan, ROC

Abstract

A set of k spanning trees rooted at the same vertex r in a graph G are called *independent spanning trees (ISTs)* if for any vertex $x \neq r$, the k paths from x to r , one path in each tree, are internally disjoint. The design of ISTs on graphs has applications to fault-tolerant broadcasting and secure message distribution in networks. It was conjectured that for any k -connected graph there exist k ISTs rooted at any vertex of the graph. The conjecture has been proved true for k -connected graphs with $k \leq 4$, and remains open otherwise. In this paper, we deal with the problem of constructing ISTs on Cartesian product of a sequence of hybrid graphs including cycles and complete graphs. Consequently, this result generalizes a number of previous works.

Keyword: independent spanning trees; Cartesian product; fault-tolerant broadcasting;

1. Introduction

Let $V(G)$ and $E(G)$ denote the vertex set and edge set of a graph $G = (V, E)$, respectively. For $x, y \in V(G)$, two paths P and Q from x to y are said to be *internally disjoint*, denoted by $P \parallel Q$, if they have no common vertex and (directed) edge except the two end vertices. A *rooted tree* is a tree with a distinguished vertex called root. A tree T in a graph G is called its *spanning tree* if T contains all vertices of G .

Let T be a spanning tree of a graph G rooted at a vertex r . For a given vertex $x \in V(T) \setminus \{r\}$, the parent of x is denoted by $\text{parent}(T; x)$, and the unique path from x to r in T is denoted by $T[x, r]$. For a graph G , two spanning trees T and T' with the same root r are called *independent spanning*

trees (ISTs, for short) provided $T[x, r] \parallel T'[x, r]$ for every vertex $x \in V(T) \setminus \{r\}$. A set of k (≥ 2) spanning trees rooted at the same vertex on a graph is *independent* if they are pairwise independent.

The design of ISTs on graphs has applications to Fault-tolerant broadcasting and secure message distribution in networks [1, 8, 15, 19]. Thus, the problem of constructing ISTs on graphs has been received much attention. However, the problem is very hard for arbitrary graphs. Zehavi and Itai [25] conjectured that for any vertex r in a k -connected graph G , there exist k ISTs of G rooted at r . The conjecture has been confirmed for k -connected graphs with $k \leq 4$ (see [8], [3, 25] and [4] for $k = 2, 3, 4$, respectively). Moreover, there are several results of ISTs on some special classes of graphs. Especially, the graph classes related to interconnection networks, such as product graphs [2, 14], planar graph [7], chordal rings [9, 21], de Bruijn and Kautz digraphs [5, 6], hypercubes [16, 24], locally twisted cubes [12, 13], folded hypercubes [19], folded hyper-stars [20], star graphs [15], multidimensional torus [17], recursive circulant graphs [22, 23], odd graphs [10] and even networks [11].

The *Cartesian product* of two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, denoted by $G_1 \times G_2$, is the graph with $V_1 \times V_2$ as its vertex set and an edge $(u_1, u_2)(v_1, v_2) \in E(G_1 \times G_2)$ if and only if either $u_1v_1 \in E_1$ and $u_2 = v_2$ or $u_2v_2 \in E_2$ and $u_1 = v_1$. In this paper, we consider the Cartesian product of a sequence of hybrid graphs including cycles and complete graphs and provide a constructing scheme of ISTs on such graphs. Let H_n denote the graph which is either an n -cycle (i.e., a cycle of n vertices) or an n -clique (i.e., a complete graph of n vertices). For the sake of convenience, we write $H_n \equiv C$ (respectively, $H_n \equiv K$) to mean that the hybrid graph is in the case of a cycle (respectively, a clique). We suppose that all vertices of H_n are labeled from 0 to $n - 1$. In particular, if $H_n \equiv C$, then two vertices are adjacent provided their labels differ by one (modulo n).

*This research was partially supported by National Science Council under the Grants NSC100-2628-E-141-001-MY2 and NSC100-2221-E-141-002.

[†]Corresponding author: shyong@mail.ntcb.edu.tw

The rest of this paper is organized as follows. Section 2 presents our constructing scheme of ISTs. Section 3 shows the correctness. The final section contains our concluding remarks.

2. Spanning trees constructing scheme

Let $\kappa(G)$ and $\delta(G)$ denote the connectivity and minimum degree of a graph G . Xu and Yang [18] provided the following lower bound of connectivity of Cartesian product graphs.

Theorem 1. [18] *Let $G_1, G_2, \dots, G_n$ be connected graphs. Then*

$$\kappa(G_1 \times G_2 \times \dots \times G_n) \geq \sum_{i=1}^n \delta_i - \max_{1 \leq i \leq n} \{\delta_i - \kappa_i\},$$

where $\kappa_i = \kappa(G_i)$ and $\delta_i = \delta(G_i)$.

Throughout the paper, we consider the graph $G = H_{m_n} \times H_{m_{n-1}} \times \dots \times H_{m_1}$, where $m_i \geq 2$ if $H_{m_i} \equiv K$ or $m_i \geq 4$ if $H_{m_i} \equiv C$ for $1 \leq i \leq n$. Clearly, G has $N = \prod_{i=1}^n m_i$ vertices and each vertex $x \in V(G)$ can be expressed as a n -tuple $(x_n, x_{n-1}, \dots, x_1)$, or simply write $x = x_n x_{n-1} \dots x_1$ for $0 \leq x_i \leq m_i - 1$, where the index i is referred to as the i -th dimension of G , and m_i is called the size of dimension i . For $1 \leq i \leq n$, if $H_{m_i} \equiv C$, then each vertex $x = x_n \dots x_{i+1} x_i x_{i-1} \dots x_1$ is adjacent to all of those vertices $x_n \dots x_{i+1} (x_i \pm 1) x_{i-1} \dots x_1$, where $x_i \pm 1$ is taken modulo m_i . On the other hand, if $H_{m_i} \equiv K$, then each vertex $x = x_n \dots x_{i+1} x_i x_{i-1} \dots x_1$ is adjacent to all of those vertices $x_n \dots x_{i+1} y_i x_{i-1} \dots x_1$, where $0 \leq x_i \neq y_i \leq m_i - 1$. For instance, Fig. 1 shows the graph $G = K_2 \times C_5 \times K_2$.

For $1 \leq i \leq n$, if $H_{m_i} \equiv C$, we let $\alpha_i = 1$ and $\beta_i = 0$, otherwise let $\alpha_i = 0$ and $\beta_i = 1$. Clearly, the graph G is vertex-symmetric and δ -regular, where $\delta = \delta(G) = \sum_{i=1}^n 2\alpha_i + (m_i - 1)\beta_i$. Also, if $H_{m_i} \equiv C$, then $\kappa(H_{m_i}) = \delta(H_{m_i}) = 2$, otherwise $\kappa(H_{m_i}) = \delta(H_{m_i}) = m_i - 1$. Thus, by Theorem 1 and the fact that $\kappa(G) \leq \delta(G)$ for any graph G , we have $\kappa(G) = \delta(G)$.

Before presenting our constructing scheme of ISTs, we need the following notation. Let $I_n = \{1, 2, \dots, n\}$ and $0 = \underbrace{00 \dots 0}_n$. Suppose that $x = x_n x_{n-1} \dots x_1 (\neq 0) \in V(G)$. Let $x(i, h)$ denote the vertex obtained from x by replacing x_i with h , where $0 \leq h \leq m_i - 1$ (i.e., $x(i, h) = x_n \dots x_{i+1} h x_{i-1} \dots x_1$). Also, let $0(i, h) = \underbrace{0 \dots 0}_{n-i} h \underbrace{0 \dots 0}_{i-1}$. We define $F_x = \{i \in I_n : x_i \neq 0\}$ and

$$\text{next}_x(i) = \begin{cases} \min\{j \in F_x : j > i\} & \text{if } i < \max F_x; \\ \min\{j \in F_x : j \leq i\} & \text{otherwise.} \end{cases}$$

That is, if $j = \text{next}_x(i)$, it means that j is the next index to the dimension i in the cyclic order such that $x_j \neq 0$.

Since G is vertex-symmetric, without loss of generality, we may consider 0 as the common root of ISTs. Moreover, since $\kappa(G) = \delta(G)$, the root in every spanning tree has a unique child. For $1 \leq i \leq n$, we denote $T_{i,h}$ as an IST such that $0(i, h)$ is the unique child of the root, where $1 \leq h \leq m_i - 1$ if $H_{m_i} \equiv K$ and $h \in \{1, m_i - 1\}$ otherwise. We now give the following algorithm to construct δ spanning trees of G . The construction can be carried out by describing the parent of every vertex in each spanning tree. For each vertex $x = x_n x_{n-1} \dots x_1 \in V(G) \setminus \{0\}$, we perform the following procedure:

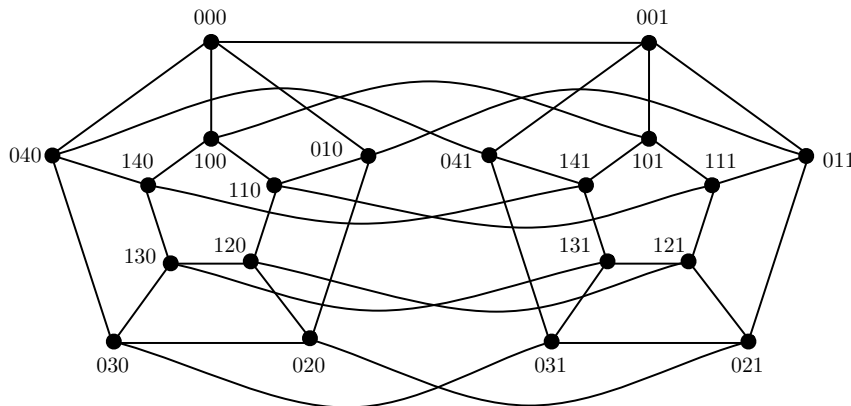


Figure 1: $G = K_2 \times C_5 \times K_2$.

Procedure Gen-Parent(x)

```

for each  $x = x_n x_{n-1} \cdots x_1 \in V(G) \setminus \{0\}$  do
    for  $i = 1$  to  $n$  do
        if  $H_{m_i} \equiv K$  then
            for  $h = 1$  to  $m_i - 1$  do
                parent( $T_{i,h}; x$ ) =  $\begin{cases} x(i, h) & \text{if } x_i \neq h; \\ \text{Do-Next}(x, i) & \text{if } x_i = h; \end{cases}$  // Case ④
            else // i.e.,  $H_{m_i} \equiv C$ 
                for  $h \in \{1, m_i - 1\}$  do
                    parent( $T_{i,h}; x$ ) =  $\begin{cases} x(i, h) & \text{if } x_i = 0; \\ x(i, x_i - 1) & \text{if } h = 1 \text{ and } x_i > \lfloor \frac{m_i}{2} \rfloor; \\ x(i, x_i + 1) & \text{if } h = m_i - 1 \text{ and } x_i \leq \lfloor \frac{m_i}{2} \rfloor; \\ \text{Do-Next}(x, i) & \text{if } h = 1 \text{ and } x_i \leq \lfloor \frac{m_i}{2} \rfloor; \\ \text{Do-Next}(x, i) & \text{if } h = m_i - 1 \text{ and } x_i > \lfloor \frac{m_i}{2} \rfloor; \end{cases}$  // Case ⑤, ⑥, ⑦, ②, ③

```

Function Do-Next(x, i)

```

 $j = \text{next}_x(i);$ 
If  $H_{m_j} \equiv K$  return  $x(j, 0);$  // Case ①
If  $H_{m_j} \equiv C$  and  $x_j \leq \lfloor \frac{m_j}{2} \rfloor$  return  $x(j, x_j - 1);$  // Case ②
If  $H_{m_j} \equiv C$  and  $x_j > \lfloor \frac{m_j}{2} \rfloor$  return  $x(j, x_j + 1);$  // Case ③

```

Note that the calculation $x_i \pm 1$ in Gen-Parent (respective, $x_j \pm 1$ in Do-Next) is taken modulo m_i (respectively, m_j). In what follows, an explicit instance can help the reader to gain an insight into the algorithm.

Example 1. We consider the construction of ISTs for $G = K_2 \times C_5 \times K_2$. Fig. 2 shows the resulting ISTs including $T_{1,1}, T_{2,1}, T_{2,4}$ and $T_{3,1}$ of G , where shadowed edges in each spanning tree indicate that Gen-Parent(x) runs into cases ①, ② and ③. In these cases, the function Do-Next is invoked and there are three subcases ①, ② and ③. Moreover, every edge is associated with a label surrounded by a circle for representing the father-son relationship. Table 1 and Table 2 provide all information for calculating the parent of each vertex $x = x_3 x_2 x_1 (\neq 0) \in G$ in $T_{2,4}$ and $T_{3,1}$, respectively. Here, we omit the corresponding tables of $T_{1,1}$ and $T_{2,1}$ since they can easily be derived from Fig. 2. $\square$

3. Correctness

In this section, we show the correctness of the constructing scheme. To explicitly represent the adjacency of vertices in G , we use the notation $x \xrightarrow{(i,h)} y$ to mean that vertex y is adjacent to vertex x such that $y = x(i, h)$. Moreover, if P and Q are two path with a common end vertex, we denote $P \circ Q$ to mean the concatenation of P and Q .

Lemma 2. For $1 \leq i \leq n$, $T_{i,h}$ is a spanning tree rooted at 0 in G where $1 \leq h \leq m_i - 1$ if $H_{m_i} \equiv K$ and $h \in \{1, m_i - 1\}$ otherwise.

Proof. It is obvious from Gen-Parents that every vertex $x (\neq 0) \in G$ is contained in $T_{i,h}$. We will show that there exists a unique path from x to 0 in $T_{i,h}$. Let $y = \text{parent}(T_{i,h}; x)$ and P be the desired path from x to 0. According to the rule in Gen-Parent, we let $j = \text{next}_x(i)$ if one of the following three conditions holds:

Case ①: $H_{m_i} \equiv K$ and $x_i = h$;

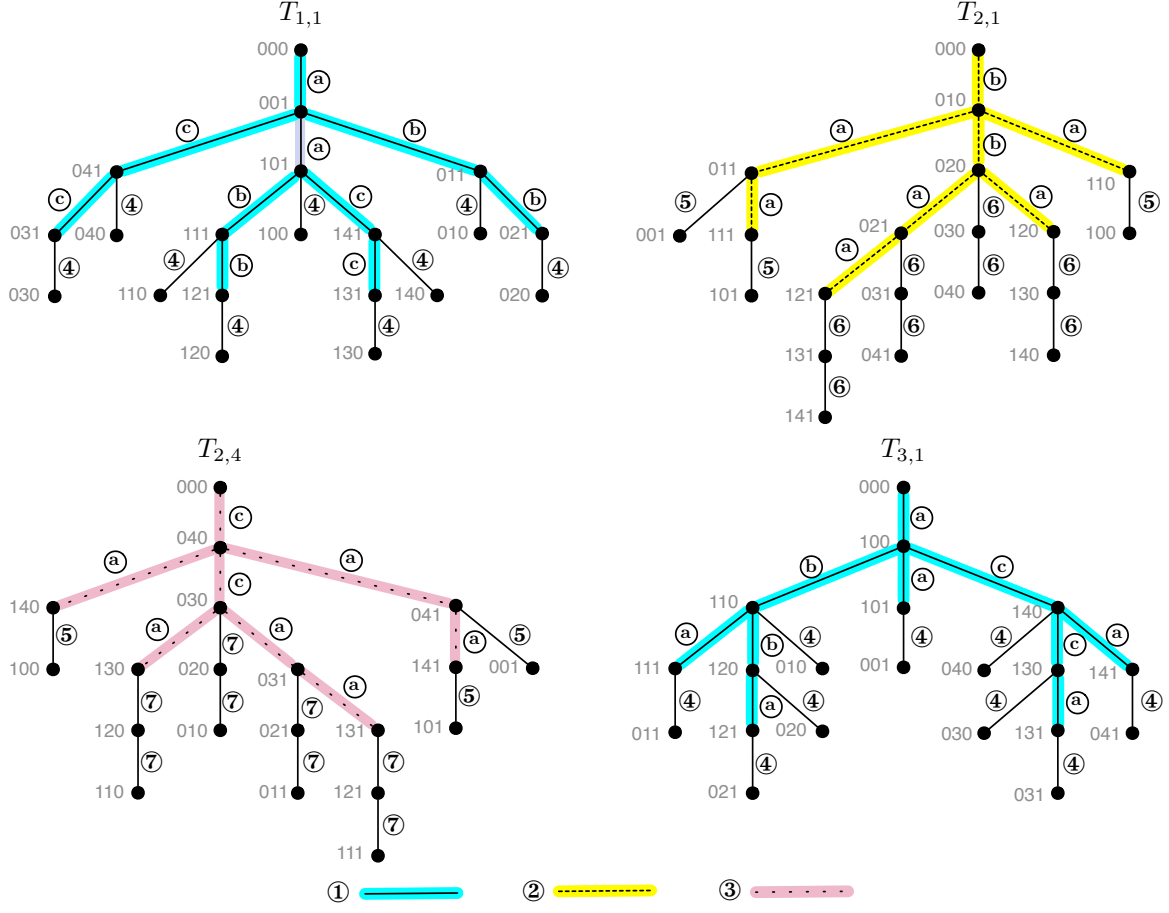
Case ②: $H_{m_i} \equiv C$, $h = 1$ and $x_i \leq \lfloor \frac{m_i}{2} \rfloor$;

Case ③: $H_{m_i} \equiv C$, $h = m_i - 1$ and $x_i > \lfloor \frac{m_i}{2} \rfloor$.

In the above three cases, we have

$$P = \begin{cases} x \xrightarrow{(j,0)} y \circ T_{i,h}[y, 0] & \text{if } H_{m_j} \equiv K \text{ (i.e., Case ①);} \\ x \xrightarrow{(j, x_j - 1)} y \circ T_{i,h}[y, 0] & \text{if } H_{m_j} \equiv C \text{ and } x_j \leq \lfloor \frac{m_j}{2} \rfloor \text{ (i.e., Case ②);} \\ x \xrightarrow{(j, x_j + 1)} y \circ T_{i,h}[y, 0] & \text{if } H_{m_j} \equiv C \text{ and } x_j > \lfloor \frac{m_j}{2} \rfloor \text{ (i.e., Case ③).} \end{cases}$$

For Case ①, if $j = i$ (i.e., $H_{m_j} \equiv K$), then y is the root of $T_{i,h}$ (i.e., $y = 0$). Otherwise, we may treat y as x and reconsider Case ① to obtain $T_{i,h}[y, 0]$. For Case ②, if $j = i$ (i.e., $H_{m_j} \equiv C$) and $x_j = 1 \leq \lfloor \frac{m_j}{2} \rfloor$, then y is the root of $T_{i,h}$. Otherwise, we may treat y as x and reconsider Case ② to obtain $T_{i,h}[y, 0]$.


 Figure 2: A set of four ISTs of $G = K_2 \times C_5 \times K_2$.

For Case ③, if $j = i$ (i.e., $H_{m_j} \equiv C$) and $x_j = m_j - 1 > \lfloor \frac{m_j}{2} \rfloor$, then y is the root of $T_{i,h}$. Otherwise, we may treat y as x and reconsider Case ③ to obtain $T_{i,h}[y, 0]$.

We now consider the remaining cases:

Case ④: $H_{m_i} \equiv K$ and $x_i \neq h$. In this case, $x \xrightarrow{(i,h)} y \circ T_{i,h}[y, 0]$ is the desired path. Then, we may treat y as x and reconsider Case ① to obtain $T_{i,h}[y, 0]$.

Case ⑤: $H_{m_i} \equiv C$ and $x_i = 0$. In this case, $x \xrightarrow{(i,h)} y \circ T_{i,h}[y, 0]$ is the desired path. Then, we may treat y as x and reconsider Cases ② and ③ to obtain $T_{i,h}[y, 0]$.

Case ⑥: $H_{m_i} \equiv C$, $h = 1$ and $x_i > \lfloor \frac{m_i}{2} \rfloor$. In this case, $x \xrightarrow{(i,x_i-1)} y \circ T_{i,h}[y, 0]$ is the desired path. Then, we may treat y as x and reconsider Cases ② and ⑥ to obtain $T_{i,h}[y, 0]$.

Case ⑦: $H_{m_i} \equiv C$, $h = m_i - 1$ and $x_i \leq \lfloor \frac{m_i}{2} \rfloor$. In this case, $x \xrightarrow{(i,x_i+1)} y \circ T_{i,h}[y, 0]$ is the desired path. Then, we may treat y as x and reconsider Cases ③ and ⑦ to obtain $T_{i,h}[y, 0]$.

Accordingly, we can check that in Cases ①, ② and ③, the label of y is obtained from x by either changing $x_j = 0$ or decreasing (respectively, increasing) x_j by one, which depends on the case of $x_j \leq \lfloor \frac{m_j}{2} \rfloor$ or $x_j > \lfloor \frac{m_j}{2} \rfloor$. Furthermore, if we move forward one step from x in one of the three cases, then all remaining steps of P will preserve the same case and without changing to other situations (i.e., all edges of P in $T_{i,h}$ have the same type for Cases ①, ② and ③). This guarantees that no two vertices of P have the same label, and thus P must be a unique path from x to 0 in $T_{i,h}$.

For Cases ④ and ⑤, after moving forward one step from x in P , the subsequent path must be the one described as in Cases ①, ② and ③. In these cases, the label on i -th dimension is changed to h and never turns back until the root. This shows that x cannot appear in the subsequent path, and thus P is a unique path.

Finally, we consider Cases ⑥ and ⑦. Since $H_{m_i} \equiv C$, we have $m_i \geq 4$ and suppose that Q is the subpath of P in the front on which it repeats the same case until $x_i \leq \lfloor \frac{m_i}{2} \rfloor$ for Case ⑥ or $x_i > \lfloor \frac{m_i}{2} \rfloor$ for Case ⑦. Then, the subsequent path

Table 1: The parent of a vertex $x(\neq 0) \in G$ in $T_{2,4}$.

x	$j = \text{next}_x(2)$	$\text{parent}(T_{2,4}; x)$	x	$j = \text{next}_x(2)$	$\text{parent}(T_{2,4}; x)$
000	-	-	100	-	140
001	-	041	101	-	141
010	-	020	110	-	120
011	-	021	111	-	121
020	-	030	120	-	130
021	-	031	121	-	131
030	2	040	130	3	030
031	1	030	131	3	031
040	2	000	140	3	040
041	1	040	141	3	041

 Table 2: The parent of a vertex $x(\neq 0) \in G$ in $T_{3,1}$.

x	$j = \text{next}_x(3)$	$\text{parent}(T_{3,1}; x)$	x	$j = \text{next}_x(3)$	$\text{parent}(T_{3,1}; x)$
000	-	-	100	3	000
001	-	101	101	1	100
010	-	110	110	2	100
011	-	111	111	1	110
020	-	120	120	2	110
021	-	121	121	1	120
030	-	130	130	2	140
031	-	131	131	1	130
040	-	140	140	2	100
041	-	141	141	1	140

must be the one described as in Cases ①, ② and ③. Clearly, any two vertices in Q have different labels on their i -th dimension and these labels are also different from those in the subsequent path. Therefore, P is a unique path from x to 0 in $T_{i,h}$. $\square$

For notational convenience, if P is a path and two vertices $x, y \in V(P)$, the subpath from x to y is denoted by $P(x, y)$. We now show the independency of the construction.

Lemma 3. *The set of spanning trees constructed from Gen-Parent is independent.*

Proof. Let $x = x_n x_{n-1} \cdots x_1 (\neq 0) \in G(V)$ be any node. For $1 \leq i, j \leq n$, let $P = T_{i,h}[x, 0]$ where $1 \leq h \leq m_i - 1$ if $H_{m_i} \equiv K$ and $h \in \{1, m_i - 1\}$ otherwise. Also, let $Q = T_{j,k}[x, 0]$ where $1 \leq k \leq m_j - 1$ if $H_{m_j} \equiv K$ and $k \in \{1, m_j - 1\}$ otherwise. We assume that P and Q are two distinct paths. The aim is to show that $P \cap Q = \{x, 0\}$. Clearly, if P or Q contains only two vertices 0 and x , the result is trivial. Hence, we assume that both P and Q contain at least three vertices. Let $y = \text{parent}(T_{i,h}; x)$ and $z = \text{parent}(T_{j,k}; x)$. Suppose that y' and z' are vertices adjacent to 0 in P and in Q , respectively. Let $b = b_n b_{n-1} \cdots b_1 \in P(y, y')$ and $d = d_n d_{n-1} \cdots d_1 \in Q(z, z')$. We will prove that there exists at least a dimension k , $1 \leq k \leq n$, such that $b_k \neq d_k$. There are two cases as follows:

Case 1: $i = j$. Since $P \neq Q$, it needs that $h \neq k$. If $H_{m_i} (= H_{m_j}) \equiv K$, from Case ④ we have $b_i = h \neq k = d_i$. On the other hand, if $H_{m_i} \equiv C$, without loss of generality, assume $h = 1$ and $k = m_i - 1$. Consider the following three cases. If $x_i \leq \lfloor \frac{m_i}{2} \rfloor$, from Cases ② and ⑦ we have $b_i \leq x_i < d_i$. If $x_i > \lfloor \frac{m_i}{2} \rfloor$, from Cases ③ and ⑥ we have $b_i < x_i \leq d_i$. Finally, if $x_i = 0$, from Case ⑤ we have $b_i = h \neq k = d_i$. Thus, $P \cap Q = \{x, 0\}$.

Case 2: $i \neq j$. Without loss of generality, we assume $j > i$. According to the rule in Gen-Parent(x), there are two cases as follows:

Case 2.1: Both y and z are adjacent to x using the rule of Cases ①, ② or ③. Let $i' = \text{next}_x(i)$. Clearly, $j \geq i' > i$. It is easy to check that $b_i = x_i$ and $b_{i'} \neq x_{i'}$. Let w be the farthest vertex from z in Q such that its label remains unchanged on the i' -th dimension. Also, let w' be the next vertex of w in Q . By the definition of w , we have $d_{i'} = x_{i'}$ for every vertex $d = d_n d_{n-1} \cdots d_1 \in P(z, w)$. Moreover, since the change of d_i is before $d_{i'}$, we have $d_i \neq x_i$ for every vertex $d = d_n d_{n-1} \cdots d_1 \in P(w', z')$. This shows that P and Q have no common vertex except 0 and x (see Figure 3).

Case 2.2: y or z is adjacent to x using the rule of Cases ④, ⑤, ⑥ or ⑦. There are two cases as follows:

Case 2.2.1: $H_{m_i} \equiv K$. By Case ④ and its succeeding rule, we have $b_i = h \neq x_i$. Let w be the far-

the vertex from z in Q such that its label remains unchanged on the i -th dimension. Also, let w' be the next vertex of w in Q . Then, we have $d_i = x_i$ for every vertex $d = d_n d_{n-1} \cdots d_1 \in Q(z, w)$ and $d_i = 0$ for every vertex $d = d_n d_{n-1} \cdots d_1 \in Q(w, z')$. Thus, $P \cap Q = \{x, 0\}$.

Case 2.2.2: $H_{m_i} \equiv C$. We may consider the following scenarios.

Case 1.2.2.1: $x_i = 0$. By Case ⑤ and its succeeding rule, we have $b_i = h$ and $d_i = 0$. Thus, $P \cap Q = \{x, 0\}$.

Case 1.2.2.2: $h = 1$ and $x_i > \lfloor \frac{m_i}{2} \rfloor$. By Case ⑥ and its succeeding rule, we have $0 < b_i < x_i$ and $d_i \geq x_i$. Thus, $P \cap Q = \{x, 0\}$.

Case 1.2.2.3: $h = m_i - 1$ and $x_i \leq \lfloor \frac{m_i}{2} \rfloor$. By Case ⑦ and its succeeding rule, we have $b_i > x_i$ and $d_i \leq x_i$. Thus, $P \cap Q = \{x, 0\}$.

In the above all cases, we have shown that $P \cap Q = \{x, 0\}$. This completes the proof. $\square$

According to Lemmas 2 and 3, we have the following theorem.

Theorem 4. *The algorithm for generating a set of δ ISTs in $G = H_{m_n} \times H_{m_{n-1}} \times \cdots \times H_{m_1}$ can be implemented in $\mathcal{O}(\delta N)$ time, where δ is the minimum degree of G and $N = \prod_{i=1}^n m_i$. In particular, the algorithm can be parallelized on G by using N processors to run in $\mathcal{O}(\delta)$ time.*

4. Concluding remarks

As we have mentioned earlier, many constructing schemes of ISTs have been proposed for some interconnection networks. It is well-known that parts of the interconnection networks are constructed by Cartesian product of graphs, e.g., hypercubes are Cartesian product of a sequence of 2-cliques, and multidimensional torus are Cartesian product of a sequence of cycles with varied lengths. Therefore, our result generalizes the previous works [2, 16, 17] and [24].

References

- [1] F. Bao, Y. Funyu, Y. Hamada, and Y. Igarashi, Reliable broadcasting and secure distributing in channel networks, in: *Proc. of 3rd International Symposium on Parallel Architectures, Algorithms and Networks, ISPAN'97*, Taipei, December 1997, pp. 472–478.
- [2] X.-B. Chen, Parallel construction of optimal independent spanning trees on Cartesian product of complete graphs *Information Processing Letters* 111 (2011) 238–235.
- [3] J. Cheriyan and S.N. Maheshwari, Finding nonseparating induced cycles and independent spanning trees in 3-connected graphs, *Journal of Algorithms* 9 (1988) 507–537.

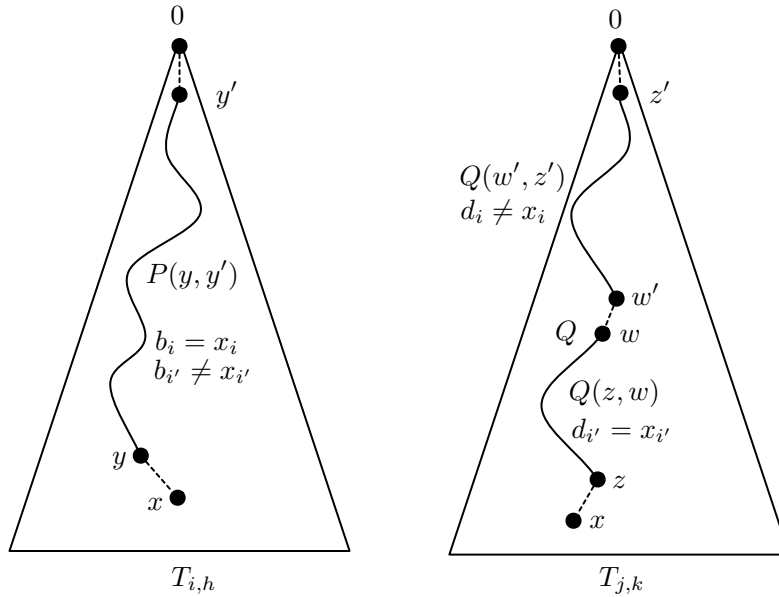


Figure 3: Illustrations of Lemma 3.

- [4] S. Curran, O. Lee, and X. Yu, Finding four independent trees, *SIAM Journal on Computing* 35 (2006) 1023–1058.
- [5] Z. Ge and S.L. Hakimi, Disjoint rooted spanning trees with small depths in deBruijn and Kautz graphs, *SIAM Journal on Computing* 26 (1997) 79–92.
- [6] T. Hasunuma and H. Nagamochi, Independent spanning trees with small depths in iterated line digraphs, *Discrete Applied Mathematics* 110 (2001) 189–211.
- [7] A. Huck, Independent Trees in Planar Graphs, *Graphs and Combinatorics* 15 (1999) 29–77.
- [8] A. Itai and M. Rodeh, The multi-tree approach to reliability in distributed networks, *Information and Computation* 79 (1988) 43–59.
- [9] Y. Iwasaki, Y. Kajiwar, K. Obokata, and Y. Igarashi, Independent spanning trees of chordal rings, *Information Processing Letters* 69 (1999) 155–160.
- [10] Jong-Seok Kim, Hyeong-Ok Lee, Eddie Cheng, László Lipták, Optimal independent spanning trees on odd graphs, *The Journal of Supercomputing* 56 (2011) 212–225.
- [11] Jong-Seok Kim, Hyeong-Ok Lee, Eddie Cheng, László Lipták, Independent spanning trees on even networks, *Information Sciences* 181 (2011) 2892–2905.
- [12] J.-C. Lin, J.-S. Yang, C.-C. Hsu, J.-M. Chang, Independent Spanning Trees vs. Edge-disjoint Spanning Trees in Locally Twisted Cubes, *Information Processing Letters* 110 (2010) 414–419.
- [13] Y.-J. Liu, J.K. Lan, W.Y. Chou, C. Chen, Constructing Independent Spanning Trees for Locally Twisted Cubes, *Theoretical Computer Science* 412 (2011) 2237–2252.
- [14] K. Obokata, Y. Iwasaki, F. Bao, and Y. Igarashi, Independent spanning trees of product graphs and their construction, *IEICE Trans. Fundamentals of Electronics, Communications and Computer Sciences*, E79-A (1996) 1894–1903.
- [15] A.A. Rescigno, Vertex-disjoint spanning trees of the star network with applications to fault-tolerance and security, *Information Sciences* 137 (2001) 259–276.
- [16] S.-M. Tang, Y.-L. Wang, and Y.-H. Leu, Optimal independent spanning trees on hypercubes, *Journal of Information Science and Engineering* 20 (2004) 143–155.
- [17] S.-M. Tang, J.-S. Yang, Y.-L. Wang and J.-M. Chang, Independent spanning trees on multidimensional torus networks, *IEEE Transactions on Computers*, 59 (2010) 93–102.
- [18] J.-M. Xu and C. Yang, Connectivity of Cartesian product graphs, *Discrete Mathematics* 306 (2006) 159–165.
- [19] J.-S. Yang, H.-C. Chan, and J.-M. Chang, Broadcasting secure messages via optimal independent spanning trees in folded hypercubes, *Discrete Applied Mathematics* 159 (2011) 1254–1263.
- [20] J.-S. Yang and J.-M. Chang, Independent spanning trees on folded hyper-stars, *Networks* 56 (2010) 272–281.
- [21] J.-S. Yang, J.-M. Chang, S.-M. Tang, and Y.-L. Wang, Reducing the height of independent spanning trees in chordal rings, *IEEE Transactions on Parallel and Distributed Systems* 18 (2007) 644–657.
- [22] J.-S. Yang, J.-M. Chang, S.-M. Tang, and Y.-L. Wang, On the independent spanning trees of recursive circulant graphs $G(cd^m, d)$ with $d > 2$, *Theoretical Computer Science* 410 (2009) 2001–2010.
- [23] J.-S. Yang, J.-M. Chang, S.-M. Tang, and Y.-L. Wang, Constructing multiple independent spanning trees on recursive circulant graphs $G(2^m, 2)$, *International Journal of Foundations of Computer Science*, 21 (2010) 73–90.
- [24] J.-S. Yang, S.-M. Tang, J.-M. Chang, and Y.-L. Wang, Parallel construction of optimal independent spanning trees on hypercubes, *Parallel Computing* 33 (2007) 73–79.
- [25] A. Zehavi and A. Itai, Three tree-paths, *Journal of Graph Theory* 13 (1989) 175–188.